

TLS-Fingerprint Scoping of Application-Layer DDoS Mitigation: Calibration Floors and Collateral on a CDN Operator’s Endpoints

Natanael Oliveira, Arthur Kobielski, Marcos Luna, Luigi Carro, Jéferson Campos Nobre, and Eder John Scheid
Institute of Informatics, Universidade Federal do Rio Grande do Sul (UFRGS), Porto Alegre, Brazil
{natanael.oliveira, arthur.kobielski, marcos.luna, carro, jcnobre, ejscheid}@inf.ufrgs.br

Abstract—A low-rate distributed application-layer flood keeps every source below per-source thresholds, so the operator’s decision is the scope of the mitigation: which clients to block, sparing legitimate users. We scope by TLS fingerprint: a binomial test names the fingerprints over-represented among an alarm’s origins against the endpoint’s normal traffic profile, at a level set to an empirical quantile of attack-free windows. On generated traffic the alarm’s most common fingerprint blocks no attacker and 39% of legitimate traffic once the botnet spans five TLS stacks, while profile-relative scopes block 90% up to 25 stacks with no observed collateral. On eight days of four endpoints of a CDN operator, the recommended configuration raises false alarms on 0.1% of clean windows, although its scope alone fires on 2.2% against a 1% target. Legitimate client fleets set a calibration floor: a 25-stack botnet on fingerprints real clients share is named only past 8% of the busiest endpoint’s window and 4 to 19 windows elsewhere, and never on those carrying most origins. Where the scope names a small botnet, the distinct-origin trigger seldom fires: only 12% of one a tenth of the busiest window is stopped. A beta-binomial background, built after a held-out day was read, nearly meets the target out of sample and lowers the floor to 6% and at most 4.4 windows. The held-out day does not contradict the false-alarm rate, and WAF verdicts prove unusable as labels. An OWL ontology specifies the counts, which its compiled query reproduces on real data.

Index Terms—security management, application-layer DDoS, mitigation policy, TLS fingerprinting, statistical calibration, ontology

I. INTRODUCTION

Application-layer Distributed Denial of Service (DDoS) attacks span two extremes. At one end, HTTP/2 Rapid Reset (CVE-2023-44487) pushed mitigation peaks to 398 million requests per second [1], a volume that capacity and scrubbing absorb. At the other end sit low-rate distributed floods, the target of this paper. *Distributed Slow HTTP DoS* and rate-based variants that still hold connections open, such as HULK and GoldenEye [2], exhaust a server from many coordinated origins, each sending requests small enough that no per-source threshold is crossed.

A stealthy campaign also shapes each session to look legitimate on the per-session signals detectors rely on, such as connection duration and route entropy [3]. What remains discriminative lives in the structure *between* sessions: the same

client fingerprint recurring across many origins that converge on one endpoint. Acting on it is a management decision: *which* clients to block, on *what* evidence, with a filter narrow enough to spare the legitimate users of the attacked service. The operator also needs to know, before an attack, where such a filter cannot be built, so that it falls back to a rate limit or a challenge.

Existing work addresses detection and leaves that decision open: 47% of 75 application-layer DDoS detectors surveyed derive their features from HTTP sessions, flattened into a numeric vector [4]. Graph-based detectors keep relations between network nodes and stop at a report [5]. Mitigations scoped to a traffic aggregate filter on network and transport headers [6], [7], source-based filtering weighs each blocked prefix against the legitimate traffic it carries [8], and the standard channels that carry a mitigation request filter on the same fields [9], [10]. None says which application clients to block, at what cost to legitimate users, or where that choice stops being possible.

This paper treats the scope of the mitigation as a calibrated statistical decision (Fig. 1). Per endpoint and five-minute window, a volume trigger raises the alarm. The scope is the set of TLS fingerprints over-represented among the alarm’s origins relative to a profile of the endpoint’s normal traffic, at a level set to an empirical quantile of attack-free windows. We target botnets of compromised devices, whose TLS stacks their operator cannot cheaply replace, and evaluate bots that present common fingerprints as the method’s boundary. This paper makes three contributions. (i) On generated traffic we show when the natural scope, the fingerprint most of the alarm shares, blocks users and no attackers, and that profile-relative scopes block the attack without labels, stacks that real clients share included. (ii) On eight days of four endpoints of a CDN operator’s own services we measure what calibration costs: the false alarms, the calibration floor at which the scope names a typical botnet stack, and a harder limit past which it names none. We also measure how much of an injected botnet the trigger lets through, and analyze a held-out day as fixed in advance. (iii) An OWL ontology specifies the counts the decision reads, and the query a log store runs is compiled from it.

TABLE I
COMPARISON OF RELATED WORK

Approach	Filter matches	Chosen by	Cost to legitimate users
Aggregate rate control [6], [7]	Header-defined aggregate	Congestion or aggregate score	Limited with the aggregate
Blackholing [11]	Destination prefix, or header fields	The attacked network	All its traffic, or less with finer filters
Source-based filtering [8]	Source prefixes	Optimization under a filter budget	Traffic of the filtered prefixes
Anomaly extraction [12], [13]	Flow feature values	Over-representation in anomalous traffic	None (diagnosis only)
Signature generation [14]	Payload tokens	Coverage against a normal pool	False positives on normal traffic
Puzzles [15]	Every client under attack	The cost a client pays	A puzzle each
DOTS, Flowspec [9], [10]	Network and transport fields	The requesting party	That of the filter carried
L7 detectors [3], [5], [16]	None (verdict only)	Profile or learned model	Detection FPR only
Bot management [17]	JA4	Undisclosed	Undisclosed
This work	TLS fingerprints (JA4)	Calibrated enrichment against the profile	Measured per alarm and window

II. RELATED WORK

Table I compares prior scopes and detectors on what a filter matches, how it is chosen and what it costs legitimate users.

Scoping a mitigation. Aggregate-based congestion control identifies the aggregate of an attack by header fields and rate-limits it upstream [6], and a recent form scores aggregates in programmable switches [7]. Blackholing drops all traffic to the attacked prefix, and advanced blackholing at an exchange point narrows the filter to header fields to limit that collateral damage [11]. Source-based filtering chooses which prefixes to block under a filter budget, trading the attack traffic stopped against the legitimate traffic of those prefixes [8], and hierarchical heavy hitters find the prefixes that carry most traffic [18]. Anomaly extraction isolates the feature values over-represented in anomalous traffic [12], [13], the question our test asks of fingerprints, and signature generation checks its candidates against a pool of normal traffic [14]. Defenses against attacks that mimic flash crowds make every client solve a puzzle [15]. All of them work on network, flow or payload features, without the fingerprints of an application’s clients. The IETF DOTS data channel installs filters on network and transport header fields [9], as BGP Flow Specification does [10], and neither can express an application-layer client fingerprint such as JA4.

Layer-7 detection. Layer-7 HTTP DDoS detectors [2] either profile traffic, applying Principal Component Analysis (PCA) to flow statistics [19] or to a per-session behavioral matrix [16], or learn from session features [3]. Neither retains a relation between sessions: whatever coordination exists is dissolved into the feature vector before the classifier sees it.

Knowledge graphs and fingerprints. Security knowledge graphs link entities extracted from threat reports, or map countermeasures to ATT&CK as D3FEND does in OWL [20], and how to apply them in operations remains unclear [21]. Graphs built from traffic feed learned detectors: KLAG [5], the closest to our setting, classifies the nodes of graphs built from network logs. Passive TLS fingerprinting identifies client software from its *ClientHello* [22], and one fingerprint is often shared by the many applications built on one library [23], which is what makes a stack a scope and what limits it. Most bots that claim a browser’s identity present the TLS and header fingerprints of another tool [24]. Industrial bot-management

platforms aggregate signals per JA4 over recent traffic [17], with no public model, benchmark or collateral metric.

Detectors thus reduce the session to a vector or keep relations between network entities in learned form, and mitigation scopes are defined over network features. No published approach derives a mitigation scope over application-client fingerprints from a calibrated test against the service’s own traffic, with its collateral damage and its limits measured. The rest of this paper does that.

III. CALIBRATED FINGERPRINT SCOPING

This section states the threat model, the trigger, the scope and its calibration, the floor calibration imposes, and what the ontology specifies (Fig. 1).

A. Threat Model

The defender operates the attacked application and observes, per request, the TLS *ClientHello* (hence a JA4 fingerprint [25]), source address and prefix/ASN, target endpoint, arrival time, and whatever identity material the application exposes. It sees neither the botnet’s control channel nor which sources are coordinated, but can profile the service’s own traffic outside attack episodes. The attacker controls K compromised devices and may shape each session to be statistically indistinguishable from a legitimate one, the stealth assumption of our evaluation, jittering, rotating or randomizing timing, identities and payloads. The damage is exhausted capacity: each session holds a connection and a worker as a legitimate one does, so no per-session rate rises. The TLS stack is harder to change: a bot that speaks TLS does so through the library its device or toolkit ships, and botnets of compromised devices span many router, camera and DVR models [26]. Presenting a browser’s *ClientHello* takes an impersonation library kept in step with browser releases [27]. Bots that do so, such as headless-browser farms or bots built on TLS-impersonation libraries, fall outside this model, and Section V evaluates them as its boundary.

B. The Trigger

Per endpoint e and five-minute window, let S be the sessions to e and n their distinct origins, $o(s)$ being the source address of session s . The trigger fires when $n \geq k_{\min} = 5$ and n reaches its 99th percentile over attack-free windows: the **distinct-origin gate** of the production evaluation. On

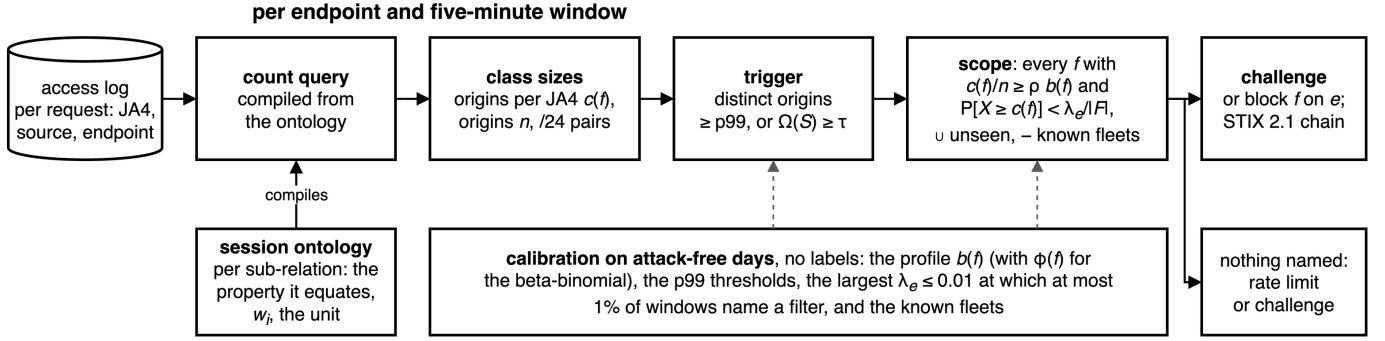


Fig. 1. Calibrated fingerprint scoping. The query compiled from the session ontology returns, per endpoint and window, the class sizes the trigger and the scope read, and every threshold and level comes from attack-free days.

generated traffic we also read the coordination mass of the ontology’s rule `CoordinatedHTTPFlood`,

$$\Omega(S) = \sum_{i \in \mathcal{R}_{\text{rel}}} w_i |E_i(S)|. \quad (1)$$

Here $E_i(S)$ is the set of pairs of distinct origins $\{o(s_a), o(s_b)\}$ whose sessions are linked by sub-relation i , w_i is its weight (Section III-E), and its threshold τ_{cluster} sits at the same percentile. Each pair counts once, so one client’s many connections cannot pose as coordination. At window scale Ω is volume: its endpoint term, $0.6\binom{n}{2}$, carries 91% of it in a canonical campaign (Appendix D), and the discriminator comes from the scope.

C. The Scope and Its Calibration

How the scope is chosen decides whether the mitigation helps or harms, and the frequency-based choice fails: the alarm’s modal fingerprint selects whatever is common, and on a service under attack that is the legitimate population. A botnet of A attackers spread uniformly over M stacks puts about $0.9A/M$ on each, the rest on one-off fingerprints, while the benign head holds a share p_1 of the n_b legitimate origins. The modal value is an attack stack only while $0.9A/M > p_1 n_b$; with the measured $p_1 = 38.4\%$ (Section IV) and a campaign as large as the legitimate traffic, only while $M < 2.3$. With more stacks the filter blocks users and no attackers.

We therefore scope against a **profile** of the endpoint’s normal traffic, kept by the operator outside attack episodes. Let $b(f)$ be the share of fingerprint f , plus $1/N$, among the profile’s N origin–fingerprint pairs, $c(f)$ the number of the alarm’s origins presenting f , and $n = \sum_f c(f)$. The **binomial enrichment test** admits every f that is enriched, $c(f)/n \geq \rho b(f)$ with $\rho = 3$, and improbable under the background, $\Pr[X \geq c(f)] < \lambda_e/|F|$ for $X \sim \text{Bin}(n, b(f))$, where the Bonferroni divisor $|F|$ counts the fingerprints of profile and alarm. The result, a *set* of fingerprints, covers a fragmented botnet. A per-fingerprint *z-score*, $z(f) = (c(f) - nb(f))/\sqrt{nb(f)(1-b(f))}$, gives a simpler profile-relative scope. A filter of fingerprints absent from the profile and seen in at least $k_{\min}$ origins (**unseen**) needs no profile statistics, and joined with the test (the **union**) it recovers new stacks too small for the level.

On generated traffic each origin carries one session, and the level stays at 0.01. On production traffic, fleets of legitimate clients switch on together and break the binomial’s independence. The level λ_e is therefore set per endpoint e to an empirical quantile: the largest value up to 0.01 at which the scope names a filter in at most 1% of the endpoint’s attack-free calibration windows, about 2.9 of a day’s 288. At levels such as 10^{-60} the binomial tail is no longer the probability of the data under the model. It is a score, whose threshold that quantile sets. The *z-score*’s threshold is set to the same budget, as the 99th percentile of each calibration window’s largest z , never below 3. Both budgets hold on the calibration windows by construction, and Section V-B measures how far they hold on the days after them.

The profile is what a defender observes in normal operation, so no attack label is involved. On production traffic it also excludes the clients the operator’s WAF blocked, since the scope runs behind the WAF. It must come from outside the attack episode, because the campaign spans the whole window and would otherwise give every candidate an enrichment of one. An adversary hiding inside a popular benign fingerprint is therefore not enriched, and the scope is refused: the correct report when no discriminator exists.

D. The Calibration Floor

A calibrated level has a price, the **calibration floor**. Among n origins the test names a stack of c only if $c/n \geq \rho b(f)$ and $\Pr[X \geq c] < \lambda_e/|F|$. The floor is the smallest botnet of A attackers on M stacks whose mean stack, $0.9A/M$ attackers in a window of $n_0 + A$ origins with n_0 the endpoint’s median window, passes both, and it grows as the level falls. It marks where the scope names a typical stack: below it the scope still names the stacks chance makes larger. Joined with the test, the unseen filter names a stack absent from the profile once it holds $k_{\min}$ origins, from $\lceil k_{\min}M/0.9 \rceil$ attackers whatever the level, 139 for $M = 25$. The level’s floor thus binds on stacks that real clients also present, and there the ratio sets a harder limit. A stack holds at most $0.9/M$ of the window, so no botnet on M stacks is enriched on a fingerprint of prevalence above $0.9/(\rho M)$, 1.2% for $M = 25$. We evaluate two remedies. The first exempts the endpoint’s *known fleets*, fingerprints named

at the nominal level in at least 5% of calibration windows, from the scope and from the calibration. The exemption is an allow-list: a bot presenting a fleet’s fingerprint is not scoped. The second is a **beta-binomial background**. Its intra-window correlation $\phi(f)$ is a moment estimate per profile fingerprint over the calibration windows, from $\text{Var}[c(f)] = nb(1-b)(1+(n-1)\phi)$, clipped to $[0, 0.99]$, and each count is tested against a beta-binomial of the same mean. A fingerprint absent from the profile keeps the binomial.

E. The Session Ontology as Specification

The ontology is the specification the operator deploys. Its central class, `ApplicationSession`, is related by `hasIdentity` to a composite `Identity`, by `originatesFrom` to its source `IPAddress`, by `targets` to an `Endpoint`, and by `mitigatedBy` to a `Mitigation`, whose scope the evidence chain carries. Six sub-properties of a symmetric `relatedTo` type the evidence that links two sessions: shared TLS fingerprint, reused identity, temporal pattern, payload signature, endpoint convergence and network proximity (Appendix A). Each carries a weight w_i ordered by the attacker’s cost of breaking the signal, from the TLS fingerprint (1.0) to network proximity (0.3), since carrier-grade NAT makes a shared /24 a poor discriminator, and weight calibration supports only the top of that order (Appendix B).

The three sub-relations exercised here are equalities: exact JA4, endpoint and /24 prefix partition the sessions into classes, so $|E_i(S)| = \sum_k \binom{n_k}{2}$ over the class sizes n_k in origins. The ontology annotates each with the session property it equates and `relatedTo` with the unit it counts in. From these annotations we compile the SQL count query a log store runs per window and endpoint, given only a binding of properties to columns. No reasoner runs in the evaluation. A new equality signal reaches the compiled query as four triples, with no change to any code (Listing 1 states the rule as SWRL and SPARQL). When the rule fires, the evidence chain (the rule, the instances, the decomposition of $\Omega(S)$ per sub-relation and the scope) is exported as JSON-LD and as STIX 2.1 [28], an *indicator* plus a *course-of-action* linked by *mitigates*.

IV. EVALUATION METHODOLOGY

Each traffic source answers one question. A generator calibrated on benign CICIDS2017 sessions and on JA4 measured at a CDN carries the controlled campaigns of the main evaluation. Laboratory captures test conventional attacks and the closest published system, and eight days of its traffic test the scopes’ false alarms and, with a botnet injected, their detection.

A. Scenarios and Datasets

The evaluation is parameterized by the *degree of distribution* K of the campaign, the number of devices the attacker controls. We report $K = 50$, a small botnet whose origins each stay inside their own rate limit, and $K = 1000$, spread over hundreds of ASNs and prefixes, with negligible per-session signal. A single source is left to volumetric detectors.

Primary evaluation: calibrated synthetic traffic. A generator produces distributed Slow HTTP DoS campaigns against one endpoint, with known coordination ground truth, parameterized by K , TLS and identity sharing, IP/ASN dispersion and the time window. Benign fingerprint popularity is shaped as a Zipf curve of exponent α : real populations are concentrated, since an up-to-date browser on a given platform emits essentially one JA4 [25]. We calibrated that curve against 6.33M TLS requests logged at a production edge point of presence of Azion Technologies¹: 495 distinct fingerprints, the most common accounting for 38.4% of requests and the top ten for 93.8%. The measured curve is bracketed by $\alpha = 1.5$ and $\alpha = 2.0$, and we take $\alpha = 1.5$ as canonical. No published measurement fixes botnet composition, so we sweep it: 90% of the campaign spreads uniformly over M stacks, the conservative choice for a given M , the rest over one-off fingerprints, and $M = 25$ is canonical. An *adversarial* mode lets the botnet adopt the most common benign fingerprints, as an impersonating bot would.

Legitimate request counts and durations are sampled from the benign sessions of CICIDS2017, all 322 658 for the first and the 40 351 with two or more requests for the second, so Kolmogorov–Smirnov distances of $D = 0.002$ and 0.003 between generated and real sessions only check the sampler. A *stealth* mode draws each attacker session’s attributes from the same distributions, so the campaign reveals itself only through cross-session structure. Attackers start within the first 30 s, so the campaign is a large share of the traffic it overlaps: a median 34–45% of a window’s origins and 50% of the alarm’s cluster. A campaign that is a small share of the window appears only in the production injection at a tenth of the window.

Laboratory captures of CICIDS2017 and CIC-IoT2023 test conventional attacks and the closest published system, KLAG (Appendix C).

Production traffic. We use eight days of four endpoints of Azion’s own services, real-user monitoring beacons (E1), a web console (E2), an API (E3) and single sign-on (E4), whose median windows hold about 3 000, 62, 41 and 20 origins. They are the four of the 12 exported that carry TLS with a median window of at least $k_{\min}$ origins. The exports count distinct origins and connections per fingerprint and five-minute window, and same-/24 pairs per window. We take the calibration days as attack-free: an attack among them would lower the level and raise the gate, making the scope more conservative. Into each window of the last five days, tested with the profile, thresholds and level of the days before them, we inject a botnet built as the generator builds one, of 25 to 1 000 attackers or of a tenth of to the whole median window. It puts 90% of its attackers on 25 stacks, uniformly, and the rest on one-off fingerprints, from 2 000 /24s disjoint from real ones. Its stacks are absent from the traffic (new), drawn from the profile past its ten most common fingerprints (shared), or the endpoint’s 25 most common fingerprints (adversarial). A

¹Measured 22 August 2026 from an access log the platform already collects. The extract holds only client-software fingerprints and their frequencies, with no address, host, URI, header or locale.

TABLE II
THE CONFIGURATIONS EVALUATED ON PRODUCTION TRAFFIC.

Configuration	Gate	Scope	Background	Exempt	Chosen	Held out
Base rule	$\Omega \geq \tau$	test	binomial	–	test days	held-out day
Binomial	origins	test $\cup$ unseen	binomial	fleets	test days	held-out day
z -score	origins	$z > \text{its p99}$	binomial	–	test days	held-out day ^a
Beta-binomial	origins	test $\cup$ unseen	beta-bin.	–	post hoc	–

Gate: $\Omega \geq \tau_{\text{cluster}}$, or distinct origins at or above their 99th percentile. Thresholds, levels and known fleets are fitted on attack-free days, in sample or cross-fitted. Five choices were made on the test days: counting origins, calibrating λ_e , the gate, the union with the unseen filter and the 5% fleet share. Post hoc: after the held-out day was read, as was the cross-fitted calibration. ^aFixed before the day was read, outside its protocol.

flash crowd adds 100 or 1000 users drawn from the day’s own clients to one window. A ninth day, the held-out day, was analyzed with the configuration fixed in advance.

B. Baselines, Configurations and Metrics

On generated traffic the scopes are compared on the alarm’s cluster: the modal fingerprint, the unseen filter, the z -score ($z > 3$) and the binomial test. The learned baseline is a Random Forest trained on the campaign’s labels over the cross-session evidence of the three sub-relations (configuration (d) of Appendix C), forced onto the test’s operating point. On production traffic the configurations are those of Table II.

Beyond classification metrics we report **collateral damage**, the share of legitimate traffic inside the scope: the false-positive rate of the filter an operator would deploy. On production traffic each configuration has a false-alarm rate, the share of clean windows where it names a filter, with an exact 95% interval and one from a bootstrap over endpoint-days, since misfires cluster by day. It also has a collateral per alarm, the share of that window’s clients the filter blocks. The rate times the mean collateral is the expected share of legitimate clients blocked per clean window. The blocked share of the injected botnet is averaged over all windows, those where the trigger does not fire included. It is the product of how often the gate fires, how often the scope names the botnet when it does, and the share of attackers the named filter covers.

V. RESULTS

The results follow the claims they support: the scopes on generated and on production traffic, the analyses built after the held-out day, and the boundary of the method.

A. The Scope on Generated Traffic

Here the sessions matching a scope *are* the flagged set, with no training or label, and the learned model is forced onto the scope’s operating point (Table III, $n = 15$ campaigns). Per-session detection sits at chance, as the stealth mode imposes (Appendix C).

The natural scope fails. For a monolithic botnet the modal fingerprint and the test agree, blocking 89.8% of the attack with no collateral observed, where the fallback, a challenge to every client, would burden every legitimate user. From five stacks on, as Section III-C predicts, the modal fingerprint is a *legitimate* one: the frequency rule blocks 0.0% of the

TABLE III
THE ENRICHMENT SCOPE OF THE LARGEST- Ω CLUSTER AS A DETECTOR.

Scenario	Binomial test			Other scopes			Learned, FPR= 0	
	Recall	FPR	F_1	Modal	z -score	Unseen	(d)	+ profile
$M=1$	89.8%	0.00%	0.946	89.8%	89.8%	89.8%	91.7%	85.9%
$M=5$	90.0%	0.00%	0.948	0.0%	90.0%	90.0%	88.6%	91.7%
$M=25$	90.3%	0.00%	0.949	0.0%	90.3%	90.3%	36.4%	87.4%
$M=100$	38.6%	0.00%	0.556	0.0%	80.4%	88.1%	17.6%	86.9%
$M=25$, shared	85.4%	2.23%	0.910	0.0%	89.9%	0.0%	–	–
$M=25$, adversarial	30.4%	3.78%	0.452	3.6%	57.3%	0.0%	7.8%	6.6%

Recall of each scope on its own. Modal: the alarm’s most common fingerprint, FPR 0% at $M=1$ (monolithic) and 39.0% elsewhere (61.1% with $\alpha = 2.0$). z -score: $z > 3$, FPR 0% except 3.37% (shared) and 11.3% (adversarial). Unseen: fingerprints absent from the profile, mean FPR 0.03% (at most 0.5% in one seed). Shared: stacks from the profile’s tail (from the generator’s vocabulary instead, the unseen filter blocks 84.4% and the test 89.8%). The learned models, which train on the campaign’s labels, skip it. $K = 1000$, $n = 15$ seeds.

attack and 39.0% of legitimate traffic, 61.1% with a more concentrated benign population ($\alpha = 2.0$).

Profile-relative scopes succeed. The binomial test and the z -score block 90.0% and 90.3% of the attack at five and 25 stacks with **no collateral observed** across the campaigns, the rest sitting on one-off fingerprints: scoped mitigation trades completeness for precision. At 100 stacks a stack of about ten sessions among two thousand cannot be told from the tail of a 1000-session profile, and the test falls to 38.6%, which a profile pooled over 30 attack-free periods restores to 89.6%. The unseen filter matches the test because generated stacks never occur in the benign vocabulary, but blocks none of the stacks drawn from the profile’s tail, which real clients also present.

The learned model needs labels that no operator holds during an attack, and at 25 stacks the test pulls ahead of it, 90.3% against 36.4% at zero false positives. Given the profile as two per-session features it recovers 87.4% and 86.9% at 25 and 100 stacks, trained on labels of the very campaign it scores. Trained on one number of stacks, (d) does not carry to another: its AUC falls from 0.93–0.98 to 0.48–0.75 (Appendix C).

Two conditions bound the result. When the botnet adopts common benign fingerprints, the boundary of the threat model, little is enriched: the test blocks 30.4% of the attack at 3.78% collateral and the frequency rule 3.6% at 39.0%. A flat or missing profile makes every fingerprint look rare, and collateral jumps to 77.6%, while moderate drift is tolerable. Per five-minute window, $\Omega(S) \geq \tau_{\text{cluster}}$ alone also fires on flash crowds [29], and requiring the test to name a fingerprint removes those false alarms (Appendix E).

B. Production Traffic

The configurations of Table II meet real clients on the four endpoints (Tables IV to VI, and Fig. 3 in Appendix E). At the nominal level the scope names a filter in 60.1% of clean windows counted in connections, since one client opens many, and in 26.9% counted in origins, because legitimate fleets switch on together. Calibrating λ_e on the preceding days brings the base rule to 0.2%, and the distinct-origin gate, which fires

TABLE IV
THE CONFIGURATIONS ON PRODUCTION TRAFFIC, ALL ENDPOINTS.

Scope	Clean windows				New stacks		Shared	
	FA	Alone	Coll.	Fl. 1k	0.1×	1×	0.1×	1×
<i>Test days, in sample</i>								
binomial	0.1	2.2	32.9	22.2	3.0	19.9	1.9	17.6
beta-bin.*	0.4	4.4	10.5	28.4	3.3	31.4	2.7	24.7
z-score	0.4	6.1	10.8	13.1	3.1	28.8	2.3	18.9
<i>Test days, cross-fitted</i>								
binomial	0.1	1.9	31.4	20.7	3.0	19.8	1.5	17.4
beta-bin.*	0.1	1.1	2.0	10.9	3.0	23.3	2.1	17.5
z-score	0.2	2.5	11.0	5.5	3.0	20.1	2.0	16.0
<i>Held-out day, calibrated on the eight days before it</i>								
rule	0.2	8.7	41.2	24.4	0.0	17.3	0.0	16.6
binomial	0.2	0.5	41.2	24.5	0.0	20.4	0.0	17.0
beta-bin.*	0.0	1.6	—	26.6	0.0	32.5	0.0	26.2
z-score	0.2	2.8	41.2	0.3	0.0	31.4	0.0	19.7

In %. FA: share of attack-free windows (5 643 on the test days, 1 152 on the held-out day) where the configuration names a filter; Alone: the same for its scope without the gate; Coll.: median share of their clients the filter blocks; Fl. 1k: FA with a flash crowd of 1 000 users. Blocked: mean share of a 25-stack botnet the size of the endpoint's median window (1×) or a tenth of it, on stacks absent from the traffic (new) or drawn from the profile past its ten most common fingerprints (shared); one-off fingerprints cap it near 90%. Per endpoint: Table VI. *Built after the held-out day was read.

TABLE V
CALIBRATION FLOOR OF THE CONFIGURATIONS.

Endpoint	Origins	λ_e		Binomial		Beta-binomial*	
		bin.	beta*	new	shared	new	shared
E1, RUM beacons	2 983	10^{-21}	10^{-4}	139/139	254/286	85/139	86/167
E2, web console	62	10^{-73}	10^{-2}	139/139	1090/1041	56/114	84/168
E3, API	41	10^{-11}	10^{-2}	139/139	168/227	56/114	56/140
E4, SSO	20	10^{-4}	10^{-2}	84/84	84/114	56/56	84/84

Origins: median per window. λ_e : calibrated level in sample, known fleets exempted under the binomial. Floor: the smallest 25-stack botnet, in attackers per window, whose stacks the configuration names, in sample / cross-fitted, medians over the five test days. On new stacks the unseen filter bounds it at 139. A shared stack takes the median prevalence past the profile's ten most common fingerprints and, under the beta-binomial, their median correlation. *Built after the held-out day was read.

TABLE VI
THE CONFIGURATIONS ON PRODUCTION TRAFFIC, PER ENDPOINT.

Scope	Clean windows				New stacks		Shared	
	FA	Alone	Coll.	Fl. 1k	0.1×	1×	0.1×	1×
E1 binomial	0.0	1.0	—	0.3	11.8	70.8	7.7	68.6
beta-bin.*	0.1	2.1	0.2	1.0	11.9	70.8	10.0	67.8
E2 binomial	0.3	4.4	31.4	20.7	0.0	5.0	0.0	0.0
beta-bin.*	1.3	10.8	10.7	74.4	0.7	25.9	0.2	13.2
E3 binomial	0.1	1.8	48.5	33.8	0.0	1.9	0.0	0.5
beta-bin.*	0.0	3.2	—	28.3	0.3	22.7	0.3	13.9
E4 binomial	0.0	1.5	—	34.1	0.1	2.1	0.0	1.5
beta-bin.*	0.1	1.4	3.5	9.8	0.1	6.0	0.1	3.9

As Table IV, test days, in sample. Coll. is a median over few alarms: 4 to 19 on E2, at most 2 elsewhere. *Built after the held-out day was read.

on the same windows as Ω in 86–99.6% of cases, halves its false alarms with no loss of detection.

False alarms. Every calibrated component exceeds its 1% target on the days after the calibration: the binomial configuration's scope alone names a filter in 2.2% of clean test-day windows, and the distinct-origin gate alone fires in 3.0%. The configuration needs both and fires on 0.1% of clean windows (5 of 5 643, 95% interval 0.03–0.21%, or 0.02–0.16% from a

bootstrap over endpoint-days), 0.25 times per endpoint and day. The two are not independent, since a fleet raises the origin count and enriches its own fingerprint at once, and they misfire together more often than independence predicts (5 windows against 3.5). Its misfires block a mean 30.6% of the window's clients, about 0.03% of legitimate clients per clean window, and a crowd of 1 000 users drawn from the day's clients triggers the configuration in 22.2% of windows. The calibrated z -score fires on 0.4% with lighter filters, its misfires clustered on one day of the console (Appendix E).

What the scope can name: the calibration floor. On the busiest endpoint fleets push λ_e to 10^{-60} , where the binomial test names a stack only past 15 to 18 origins, and exempting its five or six known fleets raises λ_e to about 10^{-21} . Joined with the unseen filter, the binomial configuration names a 25-stack botnet on new stacks from 139 attackers per window on E1 to E3, a bound $k_{\min}$ sets, and from 84 on E4 (Table V). On stacks real clients also present the level binds: it names the botnet only past 254 attackers on the busiest endpoint, 8% of its window, and past 4 to 19 times the day's median window on the small ones. The floor is no cliff: below it the scope names the stacks chance makes larger, 43% of 100 attackers on new stacks on the busiest endpoint, as a binomial model of stack sizes predicts. Past the ratio limit it names nothing: the 4 to 28 most common fingerprints of each endpoint, which carry 86–94% of its origins, are never enriched by a 25-stack botnet.

What the configuration stops: the trigger decides. Naming a botnet is not stopping it, since the distinct-origin gate must fire first. On the busiest endpoint the gate opens in 6–13% of the windows of botnets of 25 attackers to a tenth of the window. The binomial configuration therefore stops 3.4% of 100 attackers and 11.8% of a tenth of the window, against 70.8% of a whole one. A trigger on the scope alone does better (Appendix E). On the small endpoints a botnet of 100 attackers, 1.6 to 5 windows there, opens the gate, and the floor decides: the configuration stops 28–79% of it on new stacks and 2–5% of a botnet the size of the window. Pooled, it stops 38.4% and 77.7% of 100 and 1 000 attackers on new stacks and 21.7% and 63.3% on shared ones, figures the small endpoints dominate. In attack windows the calibrated scopes block a median 0% of legitimate clients and up to 71% in the worst window of a small endpoint.

The held-out day does not contradict the rate. Exported after the choices that fixed the configuration and analyzed with it and with metrics fixed in advance (Appendix F), the held-out day gives the binomial configuration 2 false alarms in 1 152 clean windows ($P = 0.27$, as up to 3 would have been). The test is weak: a tripled rate would be flagged 37% of the time, and the gate opened in only 3 clean windows, two of which the scope misfired on, both on the web console ($P = 0.003$ at the test days' rate given the gate). The base rule, the protocol's reference, fired on the same two windows, as did the calibrated z -score, outside the protocol. Flash crowds of 100 users trigger the configuration in 1.1% of windows. It stops 38.9% and 73.4% of 100 and 1 000 attackers on new stacks and 21.5%

and 60.9% on shared ones, and at a tenth of the busiest window the trigger never fired.

C. Post Hoc Analyses

The beta-binomial and a cross-fitted calibration were built after the held-out day was read, so their results are post hoc.

The beta-binomial background. Against it the busiest endpoint’s level rises to about 10^{-4} and the others’ to the 0.01 cap, and no fingerprint qualifies as a known fleet in sample. Its floor on shared stacks falls to 86 attackers on the busiest endpoint (3%) and to 1.4 to 4.2 windows elsewhere. In sample it fires on 0.4% of clean windows, with filters that block a mean 7.7% of the window’s clients, and on the held-out day it raised no false alarm.

Cross-fitted calibration. So far, the calibration windows also build the profile they are judged against. We refit each level, z threshold and set of known fleets with every calibration day judged against a profile of the other days, as a test day is. A first design, which calibrated on the last calibration day alone, was dropped because one day misses the rare fleet events that set the level. The binomial configuration barely moves, with 6 false alarms against 5. The calibrated z -score’s threshold rises by a median factor of 1.8, and its lead in detection came from the in-sample threshold. Only the beta-binomial nearly meets its target out of sample: its scope alone fires in 1.1% of clean windows. It too raises 6 false alarms, with filters that block a median 2.0% of the window’s clients, and its floor on shared stacks falls to 6% of the busiest window and at most 4.4 windows elsewhere.

D. Boundary and Labels

The boundary holds on real traffic. A botnet on the endpoint’s 25 most common fingerprints, the adversarial case, is mostly missed: the binomial configuration stops 7.0% and 23.8% of 100 and 1 000 attackers, and none on the two largest endpoints, where nearly all of those fingerprints lie past the ratio limit. Fingerprints the profile has never seen appear every day but rarely concentrate: 0.21% of the test days’ windows hold one with at least $k_{\min}$ origins.

The WAF’s verdicts cannot label floods. The operator’s WAF blocked a median 9%, 63% and 39% of each window’s clients on E2 to E4, and none on E1, most of them on fingerprints that unblocked clients also present. With the profile built without those clients, 84–99% of the clients the scopes would block on the full traffic of the API and single sign-on were blocked by the WAF. That agreement is built in: a stable fingerprint is enriched there only if 80–88% of its clients were blocked. With a profile of all clients, 0.4–33% of the clients the scopes would block were blocked, less than a random pick (41–63%). The scopes catch the WAF’s surges at chance, above it only on the console and there with under 1% of the blocked clients. The verdicts cannot validate the scope, and the scope, which reads deviations from everyday traffic, does not recover what this WAF blocks.

VI. DISCUSSION AND LIMITATIONS

We discuss which configuration to recommend and what limits the results (the limitations continue in Appendix C).

Which configuration to recommend. The floor and the trigger decide per endpoint. On the busiest endpoint every calibrated configuration names a botnet of 86 to 286 attackers per window, under a tenth of its typical window, but the distinct-origin gate lets most of it through. There the scope itself should trigger, which we examined only after the fact. On the three small endpoints the binomial configuration names shared stacks only past 4 to 19 windows and new stacks from 2 to 4 windows, so fingerprint scoping adds little and the fallback is a rate limit or a challenge to every client. Of the candidate configurations, only the binomial one was tested on a day it had not seen, as fixed in advance. We would therefore recommend it, behind the distinct-origin gate, and run the cross-fitted beta-binomial and a scope-driven trigger beside it under a pre-specified test on new days before switching. The scope should act as a challenge before a block, since a misfire hits a fleet, though non-browser clients of an API or single sign-on cannot solve one. Whether a campaign at the floor exhausts the service depends on its headroom, which we did not measure.

The gain is regime-specific. Per-IP thresholds handle single-source floods, and a strong per-session classifier reaches $AUC \geq 0.98$ on public-dataset attacks. Only distributed and stealthy campaigns need cross-session evidence, and the generated ones, a third to half of the window, are easier than the tenth at which production shows the trigger failing.

Each evaluation establishes one thing. The generator shows the mechanism under a mimicry imposed by construction, and its stacks never occur in the benign vocabulary, which is why an unseen filter matches the scope there. Production traffic measures false alarms on real clients and detection only for an injected botnet of one shape, 25 uniform stacks, over nine days of one operator’s own endpoints, too few to calibrate a weekly cycle. The five choices of Table II were made on the days they are reported on, so only the held-out day holds them out, and the cross-fitted calibration removes only the levels’ in-sample fit. At about three misfires per endpoint and day, a CDN with many endpoints needs a budget across them. None of the evaluations covers a captured stealthy campaign.

The scope depends on an observable discriminator. A bot outside the threat model that presents a common fingerprint defeats it, on generated and on production traffic. Encrypted Client Hello [30] hides JA4 from on-path observers but not from a defender that terminates TLS, as assumed here. The profile must be refreshed: a release that gives a large population a new fingerprint would reach the unseen filter at once, which the nine days did not show. Neither a flash crowd on one client type, the harder case, nor an attacker shaping the profile or the fleet exemption was injected, so profiles should be rebuilt from windows the scope did not name.

What the ontology adds, and what exchange loses. It adds nothing to the detection numbers or to the trigger: it

is the specification the operator deploys, compiled into the query a log store runs and carried into the evidence chain. That query returned the exported origin and /24-pair counts in every window of two production days, 1 152 windows each, and the JA4 class sizes in the 288 windows a day of the endpoint where the WAF blocked no client. The scope's STIX 2.1 bundles (Listing 2) pass the OASIS validator in strict mode and cross the reference TAXII 2.1 server unchanged, save the extension definition it fails to serve. MISP's importer drops the fingerprints, which STIX 2.1 carries only in an extension the importer does not map to MISP's own JA4 object (ja4-plus), and DOTS filters match only network and transport fields: fingerprint-scoped mitigation needs a standard JA4 property. Kept as classes, the equality sub-relations make admission constant and aggregation linear (Appendix D).

VII. CONCLUSION AND FUTURE WORK

We treated the scope of an application-layer DDoS mitigation as a calibrated statistical decision over the TLS fingerprints of an endpoint's clients, specified in a session ontology. On generated stealthy campaigns the alarm's most common fingerprint is a legitimate one once the botnet spans several TLS stacks, while profile-relative scopes block 90% up to 25 stacks with no observed collateral and no label. On a CDN operator's traffic, legitimate fleets set a calibration floor. The recommended binomial configuration names a botnet on fingerprints real clients share only past 8% of the busiest endpoint's window and 4 to 19 windows on the small ones, and none on the most common fingerprints. It keeps its false-alarm rate out of sample and on a held-out day, though its scope alone exceeds its target and its trigger lets most of a small botnet through. A beta-binomial background, built post hoc, nearly meets that target out of sample and lowers the floor. A pre-specified test of the beta-binomial and of triggers driven by the scope on new days, HTTP/2 attacks and a captured stealthy campaign come next.

ACKNOWLEDGMENT

We thank Azion Technologies (<https://www.azion.com/>) for authorizing the fingerprint measurement of Section IV and the aggregated traffic statistics of Appendix E, exported without addresses, ports or URIs.

REFERENCES

- [1] J. Snellman and D. Iamartino, "How it works: The novel HTTP/2 'Rapid Reset' DDoS attack," Google Cloud Blog, Oct. 2023, accessed Sep. 25, 2026. [Online]. Available: <https://cloud.google.com/blog/products/identity-security/how-it-works-the-novel-http2-rapid-reset-ddos-attack>
- [2] E. Cambiaso, G. Papaleo, G. Chiola, and M. Aiello, "Slow DoS attacks: Definition and categorisation," *International Journal of Trust Management in Computing and Communications*, vol. 1, no. 3–4, pp. 300–319, 2013.
- [3] C. Kemp, C. Calvert, T. M. Khoshgoftaar, and J. L. Leevy, "An approach to application-layer DoS detection," *Journal of Big Data*, vol. 10, no. 1, p. 22, 2023.
- [4] M. Odusami, S. Misra, O. Abayomi-Alli, A. Abayomi-Alli, and L. Fernández-Sanz, "A survey and meta-analysis of application-layer distributed denial-of-service attack," *International Journal of Communication Systems*, vol. 33, no. 18, p. e4603, 2020.
- [5] L. Belcastro, C. Carlucci, C. Cosentino, P. Liò, and F. Marozzo, "Enhancing network security using knowledge graphs and large language models for explainable threat detection," *Future Generation Computer Systems*, vol. 176, p. 108160, 2026.
- [6] R. Mahajan, S. M. Bellovin, S. Floyd, J. Ioannidis, V. Paxson, and S. Shenker, "Controlling high bandwidth aggregates in the network," *ACM SIGCOMM Computer Communication Review*, vol. 32, no. 3, pp. 62–73, 2002.
- [7] A. G. Alcoz, M. Strohmeier, V. Lenders, and L. Vanbever, "Aggregate-based congestion control for pulse-wave DDoS defense," in *Proc. ACM SIGCOMM*, 2022, pp. 693–706.
- [8] F. Soldo, K. Argyraki, and A. Markopoulou, "Optimal source-based filtering of malicious traffic," *IEEE/ACM Transactions on Networking*, vol. 20, no. 2, pp. 381–395, 2012.
- [9] M. Boucadair and T. Reddy, "Distributed Denial-of-Service Open Threat Signaling (DOTS) Data Channel Specification," Internet Engineering Task Force, RFC 8783, May 2020. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc8783>
- [10] C. Loibl, S. Hares, R. Raszuk, D. McPherson, and M. Bacher, "Dissemination of Flow Specification Rules," Internet Engineering Task Force, RFC 8955, Dec. 2020. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc8955>
- [11] C. Dietzel, M. Wichtlhuber, G. Smaragdakis, and A. Feldmann, "Stellar: Network attack mitigation using advanced blackholing," in *Proc. ACM International Conference on emerging Networking EXperiments and Technologies (CoNEXT)*, 2018, pp. 152–164.
- [12] F. Silveira and C. Diot, "URCA: Pulling out anomalies by their root causes," in *Proc. IEEE INFOCOM*, 2010.
- [13] D. Brauckhoff, X. Dimitropoulos, A. Wagner, and K. Salamatian, "Anomaly extraction in backbone networks using association rules," in *Proc. ACM Internet Measurement Conference (IMC)*, 2009, pp. 28–34.
- [14] Z. Li, M. Sanghi, Y. Chen, M.-Y. Kao, and B. Chavez, "Hamsa: Fast signature generation for zero-day polymorphic worms with provable attack resilience," in *Proc. IEEE Symp. on Security and Privacy (S&P)*, 2006.
- [15] S. Kandula, D. Katabi, M. Jacob, and A. Berger, "Botz-4-Sale: Surviving organized DDoS attacks that mimic flash crowds," in *Proc. USENIX Symp. on Networked Systems Design and Implementation (NSDI)*, 2005.
- [16] R. Bharathi and R. Sukanesh, "A PCA based framework for detection of application layer DDoS attacks," *WSEAS Transactions on Information Science and Applications*, vol. 9, no. 12, pp. 389–398, 2012.
- [17] A. Martinetti, "Advancing threat intelligence: JA4 fingerprints and inter-request signals," Cloudflare Blog, Aug. 2024, accessed Sep. 25, 2026. [Online]. Available: <https://blog.cloudflare.com/ja4-signals/>
- [18] G. Cormode, F. Korn, S. Muthukrishnan, and D. Srivastava, "Finding hierarchical heavy hitters in data streams," in *Proc. International Conference on Very Large Data Bases (VLDB)*, 2003, pp. 464–475.
- [19] G. Fernandes Jr., J. J. P. C. Rodrigues, and M. L. Proença Jr., "Autonomous profile-based anomaly detection system using principal component analysis and flow analysis," *Applied Soft Computing*, vol. 34, pp. 513–525, 2015.
- [20] P. E. Kaloroumakis and M. J. Smith, "Toward a knowledge graph of cybersecurity countermeasures," The MITRE Corporation, 2021, accessed Sep. 25, 2026. [Online]. Available: <https://d3fend.mitre.org/resources/D3FEND.pdf>
- [21] K. Liu, F. Wang, Z. Ding, S. Liang, Z. Yu, and Y. Zhou, "Recent progress of using knowledge graph for cybersecurity," *Electronics*, vol. 11, no. 15, p. 2287, 2022.
- [22] M. Husák, M. Čermák, T. Jirsík, and P. Čeleda, "HTTPS traffic analysis and client identification using passive SSL/TLS fingerprinting," *EURASIP Journal on Information Security*, vol. 2016, p. 6, 2016.
- [23] B. Anderson and D. McGrew, "TLS beyond the browser: Combining end host and network data to understand application behavior," in *Proc. ACM Internet Measurement Conference (IMC)*, 2019, pp. 379–392.
- [24] X. Li, B. Amin Azad, A. Rahmati, and N. Nikiforakis, "Good bot, bad bot: Characterizing automated browsing activity," in *Proc. IEEE Symp. on Security and Privacy (S&P)*, 2021, pp. 1589–1605.
- [25] J. Althouse, "JA4+ network fingerprinting," FoxIO Blog, Sep. 2023, accessed Sep. 25, 2026. [Online]. Available: <https://blog.foxio.io/ja4+-network-fingerprinting>
- [26] M. Antonakakis, T. April, M. Bailey, M. Bernhard, E. Bursztein, J. Cochran, Z. Durumeric, J. A. Halderman, L. Invernizzi, M. Kallitsis, D. Kumar, C. Lever, Z. Ma, J. Mason, D. Menscher, C. Seaman,

- N. Sullivan, K. Thomas, and Y. Zhou, “Understanding the Mirai botnet,” in *Proc. 26th USENIX Security Symp.*, 2017, pp. 1093–1110.
- [27] S. Frolov and E. Wustrow, “The use of TLS in censorship circumvention,” in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2019.
- [28] OASIS, “STIX version 2.1,” OASIS Standard, 2021, accessed Sep. 25, 2026. [Online]. Available: <https://docs.oasis-open.org/cti/stix/v2.1/stix-v2.1.html>
- [29] J. Jung, B. Krishnamurthy, and M. Rabinovich, “Flash crowds and denial of service attacks: Characterization and implications for CDNs and web sites,” in *Proc. 11th Int. World Wide Web Conf. (WWW)*, 2002, pp. 293–304.
- [30] E. Rescorla, K. Oku, N. Sullivan, and C. A. Wood, “TLS Encrypted Client Hello,” Internet Engineering Task Force, RFC 9849, Mar. 2026. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc9849>
- [31] M. Shafi, A. H. Lashkari, V. Rodriguez, and R. Nevo, “Toward generating a new cloud-based distributed denial of service (DDoS) dataset and cloud intrusion traffic characterization,” *Information*, vol. 15, no. 4, p. 195, 2024.
- [32] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, “Toward generating a new intrusion detection dataset and intrusion traffic characterization,” in *Proc. 4th Int. Conf. on Information Systems Security and Privacy (ICISSP)*, 2018, pp. 108–116.
- [33] E. C. P. Neto, S. Dadkhah, R. Ferreira, A. Zohourian, R. Lu, and A. A. Ghorbani, “CICIoT2023: A real-time dataset and benchmark for large-scale attacks in IoT environment,” *Sensors*, vol. 23, no. 13, p. 5941, 2023.

APPENDIX A SUB-RELATION SPECIFICATION

The equality-based sub-relations are kept as class counts (Section III-E). For the others, these procedures decide whether a candidate pair (s_a, s_b) is linked.

TLSFingerprint: exact equality of the full JA4 or, failing that, a near variant (same transport, TLS version and ALPN prefix, at most one symbol of difference in the ordered cipher or extension blocks). The variant is annotated *ja4_distance* = 1 and absorbs version drift within one client library without matching distinct stacks, at $O(1)$ per pair. **ReusedIdentity**, **TemporalPattern** and **PayloadSignature**, specified but not exercised, match a shared cookie, token or username (JWTs by their sub claim), inter-arrival sequences within a length-normalized DTW distance τ_{DTW} (FastDTW, linear), and payload summaries above a cosine similarity $\tau_{payload}$. **EndpointConvergence:** the same endpoint under path-pattern normalization (`/api/users/12345` $\rightarrow$ `/api/users/{id}`), annotated *exact* = true on an exact-path match, $O(1)$ with a prefix tree. **NetworkProximity:** a shared IPv4 /24 (IPv6 /48), a shared ASN from current BGP tables, or both (*strong* = true), $O(1)$ with an inverted prefix index.

APPENDIX B WEIGHT CALIBRATION

On laboratory captures, with a few dozen distinct JA4, optimization inverts the sign of the TLS signal, because benign sessions share fingerprints more than partially coordinated campaigns do. We calibrated the weights on the scenarios of our earlier runs, a near-uniform benign vocabulary (about 800 distinct fingerprints per 1000 benign sessions) against a monolithic botnet. Maximizing per-session attack-vs-benign ROC AUC of $w_{tls} z(ja4) + w_{ep} z(endpoint\ conv.) + w_{net} z(net\ prox.)$ over the grid

```
# SWRL (Horn): instantiate one sub-relation
ApplicationSession(?a) ^ ApplicationSession(?b) ^
differentFrom(?a,?b) ^
tlsJa4(?a,?j) ^ tlsJa4(?b,?j)
-> relatedByTLSFingerprint(?a,?b)

SELECT ?e (SUM(?w * ?n * (?n - 1) / 2) AS ?omega)
WHERE {
  { SELECT ?e ?c (COUNT(DISTINCT ?o) AS ?n)
    WHERE { ?s kg:targets ?e ; kg:inClass ?c ;
             kg:originatesFrom ?o . }
    GROUP BY ?e ?c }
  ?c kg:ofRelation ?rel .
  ?rel rdfs:subPropertyOf kg:relatedTo ;
  kg:coordinationWeight ?w .
  ?e kg:tauCluster ?tau . }
GROUP BY ?e ?tau
HAVING (SUM(?w * ?n * (?n - 1) / 2) >= ?tau)
```

Listing 1. Verdict as derivation: one Horn rule and the SPARQL aggregation.

$\{0.3 \dots 1.0\}^3$ (125 combinations, 60 scenarios, 91 500 sessions) gives ($w_{tls}=1.0$, $w_{ep}=0.3$, $w_{net}=0.3$). Only the TLS fingerprint discriminates on its own (isolated AUC = 0.93, against 0.50 for endpoint convergence and 0.58 for network proximity), and with the TLS and network weights fixed every endpoint weight of the grid attains the same optimal AUC = 0.943, as do the ontology’s weights (1.0, 0.6, 0.3). The calibration thus supports only the top of the ordering: the medium and low weights are not identifiable, and the per-window rule detects as well with uniform weights (Appendix E). The canonical scenario’s pool of 2000 benign fingerprints takes the Zipf shape of the 495 measured at one point of presence and exceeds the 76–818 each production endpoint shows. The popularity measurement is weighted per request, which if anything overstates concentration. Its head matches $\alpha = 1.5$ and its top ten $\alpha = 2.0$.

APPENDIX C ADDITIONAL LIMITATIONS

Instrumentation and deployment. Partial or noisy extraction of sessions, identities and fingerprints degrades the approach. Counting origins by source address undercounts users behind a carrier-grade NAT and overcounts clients that rotate IPv6 privacy addresses, whose natural unit is the /64. The production run is offline, over aggregated counts per endpoint, so a live CDN deployment still needs the partitioning of Appendix D, which we did not implement.

Coverage. HTTP/2 attacks and other application protocols (DNS, gRPC) need sub-relations beyond the Slow HTTP DoS family over HTTP/1.1, and $\Omega(S)$ instantiates only the three sub-relations the generator and captures expose. Identity reuse, temporal pattern and payload signature are specified (Appendix A) but not exercised, so their weights are untested.

Dataset breadth. Laboratory validation covers one dataset and one attack against KLAG, plus six attacks across CICIDS2017 and CIC-IoT2023. We found no independent capture of the stealthy regime: BCCC-cPacket-Cloud-DDoS-2024 faults the application-layer coverage of earlier datasets and ships 17 TCP-based DDoS scenarios [31].

Laboratory captures and KLAGÉ. CICIDS2017 [32] (Wednesday capture: Slowloris, Slowhttptest, HULK, GoldenEye) and CIC-IoT2023 [33] (DDoS Slowloris and HTTP Flood), on which KLAGÉ reports $F_1 = 84.1\%$ for DDoS Slowloris, run through the same extraction pipeline (PCAP $\rightarrow$ JA4 + flows $\rightarrow$ sessions). On CIC-IoT2023 DDoS Slowloris, whose attack clusters hold seven origins in one of 143 /24 prefixes, a *lean* per-session baseline (three attributes) collapses to $F_1 = 0.179$ (AUC 0.551). A *strong* one (eight flow attributes) reaches 0.900 (AUC 0.987) and the cross-session representation 0.911 (AUC 0.982), because real Slowloris, unlike our stealthy campaigns, leaves a per-session flow signature. The comparison with KLAGÉ is *uncontrolled*: our strong baseline also beats the published 0.841, and the released code, which starts from a graph of unpublished construction and ships no weights, admits no controlled rerun.

Per-session detection and learned models. Each configuration runs on $n = 30$ seeds, each campaign split 70/30 into training and test sessions. Configuration (a) uses per-session flow attributes alone, on a deliberately strong Random Forest (RF) repeated with gradient boosting, a perceptron and logistic regression. Configuration (b) adds per-session ontological attributes, (c) uses network proximity only, and (d) the cross-session evidence of the three sub-relations, the share of the cluster sharing one JA4 or /24 plus its size. We also reimplement PCA profiling [19], PCA with k -means over a behavioral matrix [16] and supervised RF/SVM over session features [3]. At $K = 50$ and $K = 1000$, (a) reaches a per-session ROC AUC of 0.498 and 0.503, (b) 0.500 and 0.502, the three academic baselines 0.495–0.518, and (a) stays at 0.488–0.503 under every classifier family ($\alpha = 1.5$, $M = 25$, $n = 30$ seeds), the chance level the stealth mode imposes. The cross-session representation (d) reaches 0.927 and 0.982, and (c), restricted to network proximity, 0.499 and 0.659 (paired Wilcoxon tests, Bonferroni-corrected, $p_{\text{Bonf}} = 7.5 \times 10^{-9}$ for (d)–(c) and (d)–(a) at $K = 1000$).

Learned representation. That separability does not transfer. Fragmenting the botnet over M stacks makes the cross-session evidence non-monotonic in the label: at $K = 1000$ an attacker shares a fingerprint with a few dozen peers, a legitimate client in the benign head with hundreds. Tree ensembles carve out the band one M produces (0.92–0.99), the perceptron reaches 0.86 and 0.95, and logistic regression, whose response to each attribute is monotone, 0.87 and 0.80 (Cohen’s $d = 13.5$ and 22.3 for (d)–(c) and (d)–(a) at $K = 1000$). Trained on five stacks and tested on 25 or 100, (d) falls to 0.61 and 0.63, and trained on 25 to 0.48 on five and 0.75 on 100, against 0.95–0.995 on the same M with other seeds. Cross- M tests use disjoint seeds, since the generator draws every benign session before anything M -dependent: a same-seed split would score (a) at 0.88–0.94 by memorization.

Explanation validation. The evidence chains of the seven laboratory clusters pass completeness (every correlated session and activated sub-relation enumerated) and actionability (a scope coherent with the cluster discriminator) checks (7/7) when counted in sessions. Counted in origins, the five of

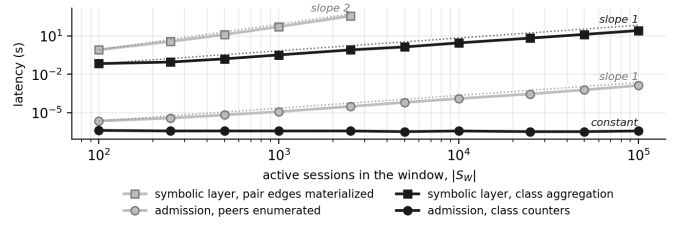


Fig. 2. Cost of both layers, with pair edges and with class counts.

CICIDS2017 hold one to three sources, below $k_{\min}$, and the two of CIC-IoT2023 hold seven. No user study tested them.

APPENDIX D COST MODEL AND WINDOW SENSITIVITY

Admission happens per request and the Ω aggregation once per window, so we time the two separately on a synthetic window: 30% of sessions in one campaign sharing a botnet JA4, benign JA4 from a pool of 2000, eight endpoints, dispersed /24s. Each runs with pair edges, as the rules are written, and with class counts for the equality sub-relations (medians of three repeats on one core), and both give the same $\Omega(S)$ to within 7×10^{-12} . The symbolic layer runs on `rdflib` in place of Jena/TDB2, so its figures are an upper bound.

With pair enumeration, median admission latency grows from $2.3 \mu\text{s}$ at $|S_W| = 100$ to $127 \mu\text{s}$ at 10 000, tracking the candidate pairs (148 to 6 495 per admission) at 15–21 ns each. With class counters it stays between 0.33 and $0.42 \mu\text{s}$ from 100 to 100 000 sessions (Fig. 2, dotted lines are reference slopes). Materializing pair edges costs 0.84 s at $|S_W| = 100$ and 52.8 s at 1 000, 198–309 μs per RDF edge. Class aggregation, one membership triple per session and sub-relation and the query of Listing 1 counting sessions (on the generator, one per origin), takes 0.33 s at 1 000 sessions, 161 times faster, and 26.4 s at 100 000. The quadratic term thus lives in the value of $\Omega(S)$, which counts pairs, and in the pair edges of the non-transitive sub-relations.

On the generator, W is the gap that splits an endpoint’s sessions into clusters. Over $W = 60$ –1800 s, on the scenarios of our earlier runs ($K = 1000$, $n = 10$), the AUC of (d) stays at 0.976–0.978 and that of (c) at 0.663–0.664. The campaign’s cluster holds 1 977–1 998 sessions at every W , while the mean cluster grows from 132 to 867, and the discriminative feature is the *fraction* of that cluster sharing one JA4, so the insensitivity is a property of the generator.

Partitioning at CDN Scale

The rule reads one endpoint per S , so partitioning by endpoint evaluates it *exactly*, with no cross-shard traffic. A campaign spreading one JA4 across many endpoints then accumulates too few pairs in any shard, and a global sketch over high-weight identifiers, left unimplemented, would need testing.

```

"@type": "kg:CoordinatedHTTFFlood",
"kg:clusterSize": 1991, "kg:coordinationScore": 1303422.8,
"kg:activatedSubRelations": [
  {"@type": "kg:relatedByTLSPFingerprint",
   "kg:coordinationWeight": 1.0, "kg:linkedPairs": 114710,
   "kg:weightedContribution": 114710.0},
  {"@type": "kg:relatedByEndpointConvergence",
   "kg:coordinationWeight": 0.6, "kg:linkedPairs": 1981045,
   "kg:weightedContribution": 1188627.0},
  {"@type": "kg:relatedByNetworkProximity",
   "kg:coordinationWeight": 0.3, "kg:linkedPairs": 286,
   "kg:weightedContribution": 85.8}],
"kg:derivedMitigationScope": {
  "kg:endpoint": "10.0.0.1:443",
  "kg:tlsJa4": ["t13d1516h2_synth_0001_00", ...]}

```

Listing 2. Evidence chain of a canonical campaign ($M = 25$, $K = 1000$), counted in origins. The scope lists 25 fingerprints, abridged.

TABLE VII
THE RULE PER FIVE-MINUTE WINDOW, 30 RUNS PER ROW.

Windows	n_W	$\Omega \geq \tau$	Ω and enrichment		
			Fires	Cov.	Coll.
Attack, $K = 1000$	90	77.8%	77.8%	79.8%	0.0%
Attack, $K = 50$	35	85.7%	85.7%	75.3%	0.0%
Clean	360	0.6%	0.0%	—	—
Flash crowd, 25	30	80.0%	0.0%	—	—
Flash crowd, 50	30	100%	0.0%	—	—
Flash crowd, 100	30	100%	0.0%	—	—

Cov.: median share of the window's attackers the scope blocks. Coll.: largest share of its legitimate sessions blocked.

APPENDIX E THE RULE PER WINDOW

Here the rule runs as deployed, per window of $W = 300$ s and endpoint, and no threshold sees an attack or a label. τ_{cluster} is the 99th percentile of Ω over the windows of 30 attack-free runs, which also form the profile. False alarms are counted on 30 more, detection on the canonical campaigns ($\alpha = 1.5$, $M = 25$) over windows with at least $k_{\min} = 5$ attackers, and a flash crowd retimes 25, 50 or 100 legitimate sessions of a third run into one window. We spread benign starts over 60 min, since the generator packs them into the first minutes.

A flash crowd adds mass as a campaign does. The test tells them apart because these crowds draw users from the benign mix the profile describes, while a surge on one client type is enriched, as production fleets show. With the test the rule fires on no clean window (one-sided 95% upper bound 0.8%) and no flash crowd, and blocks a median 75–80% of the attackers in every attack window Ω flags, hitting no legitimate session. With uniform weights Ω catches at least as many attack windows (78.9% and 85.7%), and without its endpoint term 33.3% and none, while still flagging 53–100% of flash crowds. A distinct-origin gate catches 82.2% and 85.7%, and with the test no clean window or flash crowd.

Production traffic. The exports carry no prefixes, so the scope is judged on JA4 alone. Fleets, such as a probe fleet active only in business hours, push λ_e down, and the test alone, without fleets exempted, then names a filter in 1.5–6.3% of the next day's windows. On a clean window of E2 or E3 the rule then names a fleet, blocking about half of that window's

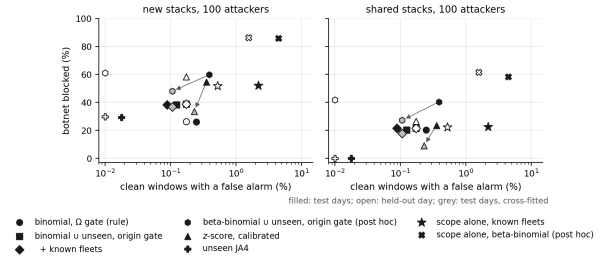


Fig. 3. Operating points on production traffic, 100 attackers per window (zero rates at 0.01%). Post hoc: the beta-binomial and the cross-fitted runs (grey).

clients, and the origin gate halves these false alarms. The 13 clean windows that only Ω admits, 8 of them with the scope firing, carry a median 33% of their Ω outside the endpoint term, against 12% elsewhere, since a fleet on one fingerprint lifts the TLS term.

Details of Section V. Gate and scope misfire together more often than independence predicts for every configuration, in sample and cross-fitted: for the beta-binomial and the calibrated z -score, 22 and 20 windows against 7.5 and 11.9. Their misfires cluster, 13 of each on one day of the console, which lifts their bootstrap upper bounds from 0.59% and 0.55% to 0.82% and 0.81%. A crowd of 1000 users, 16 to 50 typical windows of a small endpoint, triggers the calibrated z -score in 13.1% of windows. On the busiest endpoint the gate opens in 0.7–42% of the windows of a botnet a tenth of the window depending on the day. It opens in 45% at a third and in 79% at a whole window, and on the small endpoints in 14–33% at a whole window. Of 100 attackers on new stacks, the unseen filter alone stops 29.4%, the calibrated z -score 54.7% (23.7% on shared ones) and the beta-binomial 59.8% (40.5%). The shared stacks come from the profile past its ten most common fingerprints, mostly rare ones: on ranks 36 to 100 the busiest endpoint's floor rises to 743 attackers under the binomial and to 480 under the beta-binomial. Cross-fitted, the calibrated z -score stops 33.6% of 100 attackers against 54.7%. The beta-binomial's levels then fall to between 10^{-17} and 10^{-2} , its scope alone fires in 1.2% of the held-out day's windows, and one known fleet appears on the busiest endpoint on three of the five days.

Other baselines. Uncalibrated, the z -score fires on 2.6% of clean windows and 64.5% of flash crowds of 100 users (86.8% of 1000), and one against each fingerprint's own history on half of the smaller ones.

Of the four known-fleet shares tried on the first three test days, 5% was the only one whose false alarms did not exceed the base rule's (5 against 6, the others 12 to 16). On the last two it lifts enrichment on E1 at a tenth of the typical window from 0.7% to 6.3% of the attackers, at 7 false alarms against 8, and leaves E2 to E4 unchanged. Smaller shares block more but fire twice as often, on lighter filters (a median 11–12% of clients against 51%). A trigger on the scope alone fires, with known fleets, on 2.2% of pooled clean windows and 0.5% on the held-out day, against 3.5% and 8.8% without them (Fig. 3). At a tenth of the busiest endpoint's window, examined after the

fact, it stops 89.6% of the attackers on new stacks and 61.1% on shared ones at 1.0% of that endpoint's clean windows, and 90.0% and 78.3% at 2.1% with the beta-binomial. An hourly profile did not help, and $\rho = 2$ or 5 moves no pooled firing rate by over 0.3 points, no blocked share by over 4.5 and no collateral median by over 7.2. Flash crowds of 1 000 users trigger the base rule in 21.8% of windows.

APPENDIX F REPRODUCIBILITY

The reference implementation is available at <https://github.com/Natanaeloliveira/knowledge-graph-ddos-article> under the MIT license. `make compile-check stix-ingest` checks the compiled count query on generated sessions and passes the exported bundles through the OASIS STIX 2.1 validator, TAXII 2.1 and MISP's importer. The held-out day's protocol was written down, with the hashes of its exports, before the day was read (`fresh_day_protocol.md`). It was committed 17 hours after the time it states, so no external timestamp attests that order. Seeds are fixed and Makefiles drive every stage, so each table and figure regenerates from one command. The production tables (`make production-tables waf-labels`) regenerate from per-window results kept with the operator's data, and `make audit` checks every printed number against the anonymized summaries in the repository. The ontology ships in RDF/XML, and we intend to submit the artifacts to the Artifact Evaluation track.